

android-update-engine-api

Android update_engine API & OTA Package Structure — Stack Research Note

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Deep-dive on the on-device client contract for Android A/B OTA: the ota_update.zip layout, payload.bin + payload_properties.txt, the exact UpdateEngine.applyPayload(...) overloads and the four header properties (FILE_HASH / FILE_SIZE / METADATA_HASH / METADATA_SIZE), local-file:// vs HTTPS-stream apply, UpdateEngineCallback status + error-code constants with literal integer values, ZIP_STORED + HTTP Range streaming, system-UID / @SystemApi permission requirements, and ota_from_target_files generation + signing. Constants and signatures verified against primary AOSP sources. Conclusion: the Helix Android client must run with android.uid.system (or as a privileged system app) and call applyPayload with headers derived from payload_properties.txt; for verify-before-apply safety, the file:// (local) apply path is the safer mode because the artifact can be fully fetched, hashed, and signature-checked before handing the descriptor to update_engine.

Field	Value
Issues	Some ErrorCodeConstants integer values (the 4..12 range and 51/52/60/61) are read from the AOSP UpdateEngine.java mirror and error_code.h; cross-check the gaps (codes 13..50) against the Android 15 tree before quoting in code. @RequiresPermission is NOT present on applyPayload in the mirror read — access control is via @SystemApi + selinux + access to /data/ota_package/, not a manifest permission. Marked UNVERIFIED where a value could not be pinned to a single literal source.
Fixed	N/A (first revision) Follow-up should pin: (1) full ErrorCodeConstants table for the exact Android 15 (android-15.0.0_rNN) tag; (2) the IUpdateEngine AIDL/IUpdateEngineStableCallback stable surface and whether Helix should bind via the stable AIDL; (3) care_map.pb / apex_info.pb handling for Virtual A/B + APEX; (4) selinux policy (update_engine, update_engine_client domains) needed for a third-party-signed system app; (5) --payload_signer integration into the Helix Go build pipeline and AVB/vbmeta key rotation.
Continuation	

Table of Contents

- [1. Scope & Question](#)
- [2. Executive Summary](#)
- [3. OTA Package \(ota_update.zip\) Layout](#)
- [4. payload.bin + payload_properties.txt](#)
- [5. The Client API: UpdateEngine.applyPayload\(...\)](#)
- [6. The Four Header Properties](#)
- [7. Local file:/// Apply vs HTTPS-Stream Apply \(verify-before-apply\)](#)
- [8. UpdateEngineCallback: Status Constants & Error Codes](#)
- [9. ZIP_STORED + HTTP Range Streaming](#)
- [10. System-UID / Permission Requirements](#)
- [11. ota_from_target_files Generation + Signing](#)
- [12. Implications for Helix OTA](#)
- [13. Open Questions / UNVERIFIED Items](#)
- [14. Sources Consulted](#)

1. Scope & Question

Helix OTA targets **Android 15 first**, native **A/B + Virtual A/B** on device, with a **custom Go control plane** as the server. This note answers the *client/contract* questions that the broader `aosp-update-engine.md` note deferred:

- What is the exact byte/file layout of an A/B `ota_update.zip`?
- What are `payload.bin` and `payload_properties.txt`, and how do they feed the client API?
- What are the exact `applyPayload(...)` overloads, and what do the four header properties (`FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE`) mean?
- Local `file://` apply vs HTTPS streaming apply — which is safer for **verify-before-apply**?
- What are the `UpdateEngineCallback` status + error-code constants (with literal values)?
- How does `ZIP_STORED` enable HTTP-Range streaming?
- What permission / UID does the calling app need?
- How is the package generated and signed by `ota_from_target_files`?

This is the **client integration contract** that the Helix Android agent must implement, and the **build/signing contract** the Helix Go pipeline must satisfy.

2. Executive Summary

- An A/B **ota_update.zip** is a normal ZIP containing (typically): `payload.bin`, `payload_properties.txt`, `META-INF/com/android/metadata`, `META-INF/com/android/metadata.pb`, `META-INF/com/android/otacert`, `care_map.pb`, and `apex_info.pb`. The package is **whole-zip signed** (the signature lives in the ZIP comment / EOCD area) and verified against `otacert`. (Confirmed: `SystemUpdaterSample` README + AOSP OTA tools.)
- **payload.bin** is the actual update image (already internally compressed). **payload_properties.txt** is a tiny text file containing exactly four `KEY=VALUE` lines — `FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE` — generated by `system/update_engine/scripts/brillo_update_payload`. These four lines are passed verbatim as the `headerKeyValuePairs` argument to `applyPayload`.
- **UpdateEngine.applyPayload** has two overloads:
 - `applyPayload(String url, long offset, long size, String[] headerKeyValuePairs)`
 - `applyPayload(AssetFileDescriptor assetFd, String[] headerKeyValuePairs)` Plus `bind(UpdateEngineCallback) / bind(UpdateEngineCallback, Handler)` to register callbacks (`onStatusUpdate(int status, float percent)`, `onPayloadApplicationComplete(int errorCode)`).
- The URL form accepts **file://...** (local apply — `update_engine` reads the whole zip from disk) or **http(s)://...** (streaming — `update_engine` issues HTTP **Range** requests against `payload.bin` using the supplied offset/size). For streaming, all ZIP entries must be stored **uncompressed (ZIP_STORED)**.

- **Verify-before-apply: the local file:// path is the safer mode.** It lets the Helix agent download the entire artifact, verify its whole-zip signature and the payload_properties.txt hashes/sizes *before* invoking update_engine, then hand it a file:// URL (or AssetFileDescriptor). update_engine still independently verifies METADATA_HASH/FILE_HASH and the payload signature, giving defense-in-depth. Streaming verifies during/after download, so a partial/poisoned stream is only caught mid-apply.
- **UpdateStatusConstants:** IDLE=0, CHECKING_FOR_UPDATE=1, UPDATE_AVAILABLE=2, DOWNLOADING=3, VERIFYING=4, FINALIZING=5, UPDATED_NEED_REBOOT=6, REPORTING_ERROR_EVENT=7, ATTEMPTING_ROLLBACK=8, DISABLED=9 (and CLEANUP_PREVIOUS_UPDATE on Virtual A/B trees — UNVERIFIED exact value, see §8).
ErrorCodeConstants: SUCCESS=0, ERROR=1, ..., PAYLOAD_HASH_MISMATCH_ERROR=10, PAYLOAD_SIZE_MISMATCH_ERROR=11, DOWNLOAD_PAYLOAD_VERIFICATION_ERROR=12, PAYLOAD_TIMESTAMP_ERROR=51, UPDATED_BUT_NOT_ACTIVE=52, NOT_ENOUGH_SPACE=60, DEVICE_CORRUPTED=61.
- **Permissions:** android.os.UpdateEngine is @SystemApi — only system apps can use it. The practical requirement is the calling app runs as the **system UID** (android:sharedUserId="android.uid.system" + platform-signed) or is installed as a **privileged system app** in /system/priv-app/, with access to /data/ota_package/. There is **no plain manifest permission** that grants this to a normal app.
- **Generation/signing:** ota_from_target_files (in build/make/tools/releasetools) turns a target-files.zip into the OTA zip. -k/--package_key selects the whole-package signing key (default default_system_dev_certificate from META/misc_info.txt, falling back to build/target/product/security/testkey). --payload_signer controls the A/B **payload + metadata** signature (default: openssl pkeyutl with the package private key). Each key is a .x509.pem cert + .pk8 private key pair.

3. OTA Package (ota_update.zip) Layout

An Android A/B OTA package is a standard ZIP archive. The entries that matter (confirmed against the AOSP SystemUpdaterSample README and OTA tooling):

```
ota_update.zip
├─ payload.bin                # the A/B update
image (the bulk of the zip)
├─ payload_properties.txt     # 4 lines:
FILE_HASH / FILE_SIZE / METADATA_HASH / METADATA_SIZE
├─ care_map.pb               # blocks that must be
preserved/verified (Virtual A/B)
├─ apex_info.pb              # APEX module info
(when APEX present)
└─ META-INF/
    └─ com/android/
        └─ metadata          # human/parsable OTA
metadata (legacy text form)
    └─ metadata.pb          # OTA metadata
(protobuf form)
```

└─ otacert
package is verified against

the X.509 cert the

Key facts:

- The **whole zip is signed** by signapk; the signature is appended in the ZIP **central-directory / EOCD comment** region (the same “whole-file” signing scheme used for OTA packages). On device, the package signature is checked against otacert before apply. (AOSP “Sign builds for release”.)
 - META-INF/com/android/metadata(.pb) carries pre/post build fingerprints, device names, OTA type (full vs incremental), timestamps, and — for **streaming** — the **byte offsets and lengths** of the entries inside the zip, which the client turns into HTTP Range requests. (SystemUpdaterSample: “The config contains the file names and offset of the files inside the zip.”)
 - payload.bin is **already internally compressed** by the payload generator, so storing the zip entries uncompressed (ZIP_STORED, see §9) costs almost nothing.
-

4. payload.bin + payload_properties.txt

payload.bin is the binary consumed by update_engine’s payload consumer. It begins with a **payload metadata** region (magic CrAU, header + manifest + metadata signature) followed by the operation data blobs. update_engine reads/verifies the **metadata** first (sized by METADATA_SIZE, hashed by METADATA_HASH), then streams/applies the rest (sized by FILE_SIZE, hashed by FILE_HASH). (AOSP payload_metadata.{h,cc}.)

payload_properties.txt is produced by system/update_engine/scripts/brillo_update_payload. It contains exactly the four properties, e.g. (literal example from AOSP/community sources):

```
FILE_HASH=lURPCiKIAjtm0yB/EjQcl8zDzqtD6Ta3tJef6G/+z2k=  
FILE_SIZE=871903868  
METADATA_HASH=tBvj43Q0B0Jn++JojcpVdbRLz0qdAuL+uTkSy7hokaw=  
METADATA_SIZE=70604
```

- *_HASH values are **SHA-256, base64-encoded**.
- *_SIZE values are **bytes** (decimal).
- These four lines are read by the client and passed **verbatim** as the String[] headerKeyValuePairs argument to applyPayload.

The Helix Go build pipeline must surface these four values to the control plane (or let the agent read payload_properties.txt directly from the package), because the agent cannot apply without them.

5. The Client API:

UpdateEngine.applyPayload(...)

From AOSP frameworks/base/core/java/android/os/UpdateEngine.java (@SystemApi). Verified overloads:

```
// Streaming or local-by-URL form
public void applyPayload(String url,
                        long offset,
                        long size,
                        String[] headerKeyValuePairs)

// File-descriptor form (Android 8.0+/AssetFileDescriptor variant)
public void applyPayload(@NonNull AssetFileDescriptor assetFd,
                        @NonNull String[] headerKeyValuePairs)
```

Callback registration:

```
public boolean bind(final UpdateEngineCallback callback, final
                  Handler handler)
public boolean bind(final UpdateEngineCallback callback)
```

UpdateEngineCallback (verified against UpdateEngineCallback.java):

```
public abstract void onStatusUpdate(int status, float
                                   percent);           // status ∈ UpdateStatusConstants
public abstract void onPayloadApplicationComplete(int
                                                  errorCode); // errorCode ∈ ErrorCodeConstants
```

Typical call flow for the Helix agent:

1. UpdateEngine engine = new UpdateEngine();
2. engine.bind(callback);
3. Read the 4 lines from payload_properties.txt into String[] props.
4. Compute offset/size of the payload.bin entry within the zip.
5. engine.applyPayload(url, offset, size, props);
 - url = "file:///data/ota_package/ota_update.zip" for local apply,
 - OR
 - url = "https://cdn.example/ota_update.zip" for streaming.
6. Drive UI/telemetry from onStatusUpdate; finalize on onPayloadApplicationComplete(SUCCESS) then reboot.

Note on offset/size: for both the file:// and https:// URL forms, offset/size describe the **payload.bin entry's position inside the outer zip** (not the whole zip). update_engine reads only that byte range as the payload.

6. The Four Header Properties

Property	Type	Meaning	Source
FILE_HASH	base64(SHA-256)		payload_properties.tx

Property	Type	Meaning	Source
		Hash of the entire payload.bin ; update_engine rejects with PAYLOAD_HASH_MISMATCH_ERROR (10) on mismatch	
FILE_SIZE	bytes (decimal)	Total length of payload.bin; mismatch → PAYLOAD_SIZE_MISMATCH_ERROR (11)	payload_properties.tx
METADATA_HASH	base64(SHA-256)	Hash of the payload metadata prefix (first METADATA_SIZE bytes); verified early , before downloading the rest	payload_properties.tx
METADATA_SIZE	bytes (decimal)	Length of the metadata prefix region of payload.bin	payload_properties.tx

METADATA_HASH/METADATA_SIZE exist so the engine can validate the small leading metadata region **before** committing to the (potentially huge) full download — important for the streaming path where you want to fail fast. The payload’s own embedded metadata signature (signed by the payload key, see §11) is verified in addition to these hashes.

7. Local file:// Apply vs HTTPS-Stream Apply (verify-before-apply)

Both modes go through the same applyPayload; the difference is the url scheme.

Aspect	Local (file://)	Streaming (https://)
What downloads the bytes	The Helix agent downloads the full zip to /data/ota_package/, then points update_engine at it	update_engine itself downloads via HTTP Range as it applies
Disk usage	Needs room for the whole package on /data	Minimal extra storage (streams into the inactive slot)
Verify-before-apply	Strong — agent can verify whole-zip signature and the FILE_HASH/METADATA_HASH of payload.bin before calling applyPayload	Weaker — engine verifies METADATA_HASH early and FILE_HASH as data arrives; a corrupted/MITM’d stream is only caught during apply
Network requirement during apply	None (already on disk)	Live connection for the whole apply; CDN must support Range + ZIP_STORED
Resumability	Agent controls download/retry policy	Engine resumes the HTTP transfer

Recommendation for Helix (verify-before-apply): prefer the **local file://** (or `AssetFileDescriptor`) path. The Helix agent should:

1. Download the full `ota_update.zip` to `/data/ota_package/`.
2. Verify the **whole-zip signature** against the Helix-pinned otacert/cert chain.
3. Independently verify `FILE_HASH / FILE_SIZE / METADATA_HASH / METADATA_SIZE` from `payload_properties.txt`.
4. Only then call `applyPayload(file://..., offset, size, props)`.

`update_engine` then re-verifies metadata + payload signature, giving two independent verification layers (agent-level + engine-level) — a defense-in-depth posture consistent with Helix’s Uptane/TUF-style supply-chain goals. Streaming remains an option for storage-constrained devices but trades away the pre-apply full-artifact verification window. (*Engineering assessment — HIGH confidence on the mechanics, MEDIUM on it being the universally “right” choice for every Helix device class.*)

8. UpdateEngineCallback: Status Constants & Error Codes

UpdateEngine.UpdateStatusConstants (verified integer values)

Constant	Value
IDLE	0
CHECKING_FOR_UPDATE	1
UPDATE_AVAILABLE	2
DOWNLOADING	3
VERIFYING	4
FINALIZING	5
UPDATED_NEED_REBOOT	6
REPORTING_ERROR_EVENT	7
ATTEMPTING_ROLLBACK	8
DISABLED	9
CLEANUP_PREVIOUS_UPDATE	UNVERIFIED — present on Virtual A/B trees; confirm value (commonly 12 in <code>update_status.h</code>) for the Android 15 tag

The Java constants are required by AOSP comments to agree with `system/update_engine/client_library/include/update_engine/update_status.h`.

UpdateEngine.ErrorCodeConstants (verified where literal-sourced)

Constant	Value	Notes
SUCCESS	0	apply succeeded
ERROR	1	generic error
FILESYSTEM_COPIER_ERROR	4	
POST_INSTALL_RUNNER_ERROR	5	post-install script failed
PAYLOAD_MISMATCHED_TYPE_ERROR	6	full vs incremental mismatch
INSTALL_DEVICE_OPEN_ERROR	7	
KERNEL_DEVICE_OPEN_ERROR	8	
DOWNLOAD_TRANSFER_ERROR	9	network/transfer failure (streaming)
PAYLOAD_HASH_MISMATCH_ERROR	10	FILE_HASH mismatch
PAYLOAD_SIZE_MISMATCH_ERROR	11	FILE_SIZE mismatch
DOWNLOAD_PAYLOAD_VERIFICATION_ERROR	12	payload signature/verification failed
PAYLOAD_TIMESTAMP_ERROR	51	target build older than current (anti-rollback)
UPDATED_BUT_NOT_ACTIVE	52	applied but slot not made active
NOT_ENOUGH_SPACE	60	insufficient space (Virtual A/B COW)
DEVICE_CORRUPTED	61	

Codes between 13 and 50 (e.g. metadata-signature errors) exist in the underlying system/update_engine/common/error_code.h C++ enum but are not all surfaced as named Java ErrorCodeConstants. Relevant C++ values verified from error_code.h:

error_code.h enum	Value
kSuccess	0
kError	1
kPayloadHashMismatchError	10
kPayloadSizeMismatchError	11
kDownloadPayloadVerificationError	12

error_code.h enum	Value
kDownloadPayloadPubKeyVerificationError	18
kDownloadMetadataSignatureError	24
kDownloadMetadataSignatureVerificationError	25
kDownloadMetadataSignatureMismatch	26
kDownloadOperationHashVerificationError	27
kDownloadOperationHashMissingError	38
kDownloadMetadataSignatureMissingError	39
kPayloadTimestampError	51
kUpdatedButNotActive	52
kNotEnoughSpace	60
kDeviceCorrupted	61

The Java ErrorCodeConstants are required by AOSP comments to agree with system/update_engine/common/error_code.h.

9. ZIP_STORED + HTTP Range Streaming

For the streaming apply path:

- **Every ZIP entry in the OTA package must be stored uncompressed (ZIP_STORED),** not ZIP_DEFLATED. SystemUpdaterSample README: *“ZIP entries in such a package need to be saved uncompressed (ZIP_STORED), so that their data can be downloaded directly with the offset and length.”*
- Because payload.bin is already internally compressed by the payload generator, storing it ZIP_STORED adds only marginal size: *“Since payload.bin itself is already in compressed format, the size penalty is marginal.”*
- The streaming metadata (in META-INF/com/android/metadata(.pb)) gives the **offset and length** of each entry inside the outer zip. The client passes payload.bin’s offset/size into applyPayload(url, offset, size, props), and update_engine issues **HTTP Range: bytes=offset-(offset+size-1)** requests directly against payload.bin — downloading only the bytes it needs rather than the whole package.
- **Server/CDN requirement:** the host serving ota_update.zip over HTTPS must support **HTTP Range requests** (Accept-Ranges: bytes, 206 Partial Content). Compressing the zip on the wire (transfer encoding) would break byte-range offsets, so serve it as-is.

This is exactly the constraint the Helix Go control plane / CDN must honor: build packages with ZIP_STORED, publish per-entry offsets/sizes, and serve over a Range-capable HTTPS endpoint.

10. System-UID / Permission Requirements

- `android.os.UpdateEngine` and `UpdateEngineCallback` are annotated **@SystemApi** — “only system apps can access them.” They are **not** in the public SDK.
 - There is **no @RequiresPermission / manifest permission** on `applyPayload` in the AOSP mirror that a normal third-party app could request. Access is gated by being a **system-privileged caller** + `selinux`.
 - Practical requirements for the Helix Android agent:
 1. Run as the **system UID** via `android:sharedUserId="android.uid.system"` and sign with the **platform key**, OR be installed as a **privileged system app** under `/system/priv-app/` (also `platform/OEM-signed`).
 2. Have read/write access to `/data/ota_package/` (used for local `file://` apply and where `update_engine` expects packages).
 3. Bind to the `update_engine` daemon over Binder/AIDL (the `UpdateEngine` class wraps `IUpdateEngine`); `selinux` policy must allow the app’s domain to talk to the `update_engine` service.
 - Because `platform-signing / /system/priv-app/` placement is an **integrator/OEM** capability, the Helix agent must be shipped as part of the system image (or via an OEM partnership), not as a Play-Store app. *(This matches the broader `aosp-update-engine.md` “wrap, don’t replace” conclusion.)*
-

11. ota_from_target_files Generation + Signing

`ota_from_target_files` (`build/make/tools/releasetools/ota_from_target_files.py`) converts a build’s **target-files.zip** into the OTA package.

Generation:

- **Full OTA**: produced from a single `target-files.zip` (contains everything for the target slot).
- **Incremental/delta OTA**: produced by diffing against a `PREVIOUS-target-files.zip` (smaller payload).
- The tool calls into the `update_engine` payload generator (`delta_generator / brillo_update_payload`) to produce `payload.bin` and writes `payload_properties.txt` with the four hash/size values (§4).
- For **streaming**, the tool produces a package whose entries are `ZIP_STORED` and emits the per-entry offset/size streaming metadata.

Signing (two distinct signatures):

1. **Whole-package signature** — selected by `-k / --package_key`.
 - Default: `default_system_dev_certificate` from the input `target-files’ META/misc_info.txt`; if unset, falls back to `build/target/product/security/testkey`.
 - Performed by `signapk` (whole-file/zip-comment signing). On device, verified against `otacert`.
2. **A/B payload + metadata signature** — controlled by `--payload_signer`.
 - Default: signs the payload and metadata by calling `openssl pkeyutl` with the package private key.

- This is the signature `update_engine` checks via `METADATA_HASH` / payload pubkey verification (errors 18/24/25/26 in §8).

Key material:

- Each key = a **.x509.pem** certificate + a **.pk8** private key. The **.pk8** must be kept secret. (AOSP “Sign builds for release”.)
 - **For Helix:** the Go build pipeline must (a) run `ota_from_target_files` with `ZIP_STORED` + streaming metadata, (b) sign the package with a Helix-controlled `--package_key` whose cert is embedded as `otacert` and pinned on device, and (c) sign the payload via `--payload_signer`. Key rotation and HSM-backed `--payload_signer` integration are a follow-up (see Continuation row).
-

12. Implications for Helix OTA

- **Client contract is small and stable.** Helix’s Android agent = `bind()` + `read_payload_properties.txt` + `applyPayload(...)` + `handle_onStatusUpdate/onPayloadApplicationComplete`. Map the status/error constants in §8 directly into Helix telemetry codes.
 - **Ship as a system component.** The agent must be platform-signed and run as `android.uid.system` (or `/system/priv-app/`). This is non-negotiable for `@SystemApi` access and confirms Helix can’t be a normal app.
 - **Prefer local file: // apply for verify-before-apply.** Download → verify whole-zip signature + the four hashes → `applyPayload(file://...)`. Streaming is an opt-in for storage-constrained fleets, with the CDN supporting `Range` + `ZIP_STORED`.
 - **Build pipeline owns two signatures.** `--package_key` (otacert-pinned) and `--payload_signer`. The four `payload_properties.txt` values must flow from the build into the control-plane release metadata so the agent (and the control plane) can pass/validate them.
 - **Anti-rollback is enforced by the engine** via `PAYLOAD_TIMESTAMP_ERROR` (51); Helix’s rollout policy should never try to push an older build to a newer device — the engine will refuse it.
-

13. Open Questions / UNVERIFIED Items

- `CLEANUP_PREVIOUS_UPDATE` status constant value on the **Android 15** tree — UNVERIFIED (commonly 12 in `update_status.h`; confirm).
- The full set of `ErrorCodeConstants` exposed in Java for **Android 15** (codes 13..50 surface inconsistently between the C++ `error_code.h` enum and the Java constants) — needs confirmation against `android-15.0.0_rNN`.
- Whether Helix should bind via `IUpdateEngine` or the **stable AIDL** `IUpdateEngineStable` / `IUpdateEngineStableCallback` surface (relevant for forward-compat across Android versions) — UNVERIFIED, needs confirmation.
- Exact selinux domains/policy needed for a Helix-signed system app to reach `update_engine` — not researched here.
- `care_map.pb` / `apex_info.pb` exact handling for Virtual A/B + APEX during streaming — deferred.

14. Sources Consulted

Primary (AOSP source / official docs):

- `UpdateEngine.java` (frameworks/base) — `applyPayload` overloads, `bind`, `@SystemApi`, header example: <https://android.googlesource.com/platform/frameworks/base/+/master/core/java/android/os/UpdateEngine.java> and AOSP mirror https://github.com/aosp-mirror/platform_frameworks_base/blob/master/core/java/android/os/UpdateEngine.java and Code Search <https://cs.android.com/android/platform/superproject/+/master:frameworks/base/core/java/android/os/UpdateEngine.java>
- `UpdateEngineCallback.java` (frameworks/base) — callback methods & constants references: https://github.com/aosp-mirror/platform_frameworks_base/blob/master/core/java/android/os/UpdateEngineCallback.java
- `error_code.h` (system/update_engine) — C++ `ErrorCode` enum values: https://android.googlesource.com/platform/system/update_engine/+/master/common/error_code.h
- `payload_metadata.{h,cc}` (system/update_engine) — payload metadata structure: https://cs.android.com/android/platform/superproject/+/master:system/update_engine/payload_consumer/payload_metadata.h and https://android.googlesource.com/platform/system/update_engine/+/master/payload_consumer/payload_metadata.cc
- `payload_properties.{h,cc}` (system/update_engine) — properties generation: https://android.googlesource.com/platform/system/update_engine/+/refs/tags/aml_cbr_330810000/payload_generator/payload_properties.h
- `brillo_update_payload` script — produces `payload_properties.txt`: https://android.googlesource.com/platform/system/update_engine/+/HEAD/scripts/brillo_update_payload
- `SystemUpdaterSample` README (bootable/recovery/updater_sample) — package layout, streaming vs non-streaming, `ZIP_STORED`, system-app requirement: https://android.googlesource.com/platform/bootable/recovery/+/master/updater_sample/README.md
- `ota_from_target_files.py` (build/make/tools/releasetools) — generation + signing options: https://github.com/aosp-mirror/platform_build/blob/master/tools/releasetools/ota_from_target_files.py and https://android.googlesource.com/platform/build/+/master/tools/releasetools/ota_from_target_files.py
- “A/B (seamless) system updates” — Android Open Source Project: <https://source.android.com/docs/core/ota/ab>
- “Sign builds for release” — Android Open Source Project (`.x509.pem/.pk8`, package signing): https://source.android.com/docs/core/ota/sign_builds

Secondary (community, used only to corroborate, not as primary truth):

- `chendongqi/ab_ota_update_sample` (`UpdateManager.java`): https://github.com/chendongqi/ab_ota_update_sample
 - `chenxiaolong/Custota` (third-party A/B OTA updater for custom servers): <https://github.com/chenxiaolong/Custota>
 - `Rawsec` — “Android OTA payload dumping / extraction” (package internals): <https://blog.raw.pm/en/android-OTA-payload-dumping/>
-

15. Confidence

Overall confidence: HIGH for the API surface (applyPayload overloads, bind, callback signatures), the four header properties and their semantics, the OTA package layout, ZIP_STORED + Range streaming, the @SystemApi / system-UID requirement, and the ota_from_target_files --package_key / --payload_signer signing model — all read from primary AOSP source files and official docs.

MEDIUM for the complete ErrorCodeConstants integer table on the exact **Android 15** tag (gaps 13..50 not all enumerated; values quoted are from the master mirror and error_code.h) and for CLEANUP_PREVIOUS_UPDATE's exact value — flagged UNVERIFIED in §8/§13.

MEDIUM for the engineering recommendation that local file:// apply is the “right” default for Helix (the *mechanics* of why it's safer for verify-before-apply are HIGH confidence; the blanket recommendation depends on Helix device storage constraints).